高级语言程序设计
实验报告

南开大学 计算机学院
姓名 母焯行
学号 2513960
班级 计算机伯苓 2 班
2026 年 5 月 11 日

目录

高级语言程序设计大作业实验报告 1

一. 作业题目 1

二. 开发软件 1

三. 课题要求 1

四. 主要流程 1

1. 整体流程 2

2. 算法或公式 2

2.1 随机初始状态生成

2.2 阴影放置

2.3 提示算法

2.4 撤销功能

2.5 计时与最佳记录

五. 测试 6

六. 收获 8

一. 作业题目

数字华容道（Digital Sliding Blocks）

实现一款图形界面的数字华容道游戏，支持自定义棋盘大小、多级难度（含阴影格机制）、盲棋模式、计时与最佳记录、撤销与提示等功能。

二. 开发软件

・开发环境：Visual Studio / Code::Blocks（支持 C++11 及以上）

・图形库：EGE（Easy Graphics Engine，EasyX 风格扩展库）

・编程语言：C++

・辅助工具：Windows 控制台 / 图形窗口

三. 课题要求

1) 面向对象。

2) 单元测试。

3) 模型部分

4) 验证

四. 主要流程

1. 整体流程

程序启动 → 初始化图形窗口 → 进入主菜单

用户选择棋盘大小（3~8）→ 选择难度（1~6 或 0 盲棋）

→ 初始化游戏

游戏主循环:

· 绘制当前棋盘状态 (根据普通/盲棋/阴影模式不同样式)

· 检测键盘输入 (WASD、U、R、P、H、Q 等)

· 执行移动、撤销、重置、洗牌、提示等逻辑

· 每步后检查是否胜利，胜利则停止计时、保存最佳成绩、显示特效

· 按 Q 返回菜单

盲棋模式特殊流程:

· 观察阶段：显示所有真实数字→按 P 开始复原

· 复原阶段：所有格子显示紫色“NKU”→用户凭记忆移动→按 P 提交

· 提交后判断是否与正确答案一致，一致则胜利并记录三段用时；否则失败并显示真实盘面。

2. 算法或公式

2.1 随机初始状态生成

使用 随机移动法：从标准顺序状态开始，执行 500 次随机交换（移动空白格相邻块），保证最终状态有解。

```
void generateRandomState() {
   board.assign(N, vector<int>(N, 0));
   int idx = 1;
   for (int i = 0; i < N; ++i)
   for (int j = 0; j < N; ++j)
       if (i == N - 1 && j == N - 1) board[i][j] = 0;
          else board[i][j] = idx++;
    int moves = 500;
   int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
   for (int m = 0; m < moves; ++m) {
       int br, bc;
       findBlank(br, bc);
       vector<int> dirs;
       for (int d = 0; d < 4; ++d) {
          int nr = br + dr[d], nc = bc + dc[d];
          if (nr >= 0 && nr < N && nc >= 0 && nc < N)
dirs.push_back(d);
       }
       if (dirs.empty()) continue;
       int d = dirs[rand() % dirs.size()];
       int nr = br + dr[d], nc = bc + dc[d];
       swap(board[br][bc], board[nr][nc]);
   }
   placeShadows();
   initialBoard = board;
   initialShadow = hasShadow;
}
```

2.2 阴影放置

根据难度计算阴影数量（百分比范围）：

难度 2：10%~20% 格子

难度 3：30%~40%

难度 4：50%~60%

难度 5：70%~80%

难度 6：N^2-1 个阴影（仅剩一个非阴影）

随机打乱所有非空格位置，选取前 shadowCount 个标记为阴影。

```
shuffle(pos.begin(), pos.end(),
mt19937{random_device{}()});
for (int k = 0; k < shadowCount; ++k) hasShadow[r][c] = true;
```

2.3 提示算法

启发式部分：计算曼哈顿距离总和，尝试移动空白格邻近的四个方向，选择使总距离下降最多的方向（贪心）。

精确部分：若贪心未能找到最优（或曼哈顿距离仍不为 0），则启动 广度优先搜索（BFS），状态用字符串编码并存储访问集。

BFS 最多运行 2 秒（HINT_CALC_MAX_TIME），超时则返回贪心方向，否则返回 BFS 找到的第一步方向。

```
int calcManhattan(const vector<vector<int>>& b) {
   for each cell if val != 0:
   targetR = (val-1)/N, targetC = (val-1)%N;
   sum += abs(i-targetR) + abs(j-targetC);
   return sum;
}
```

2.4 撤销功能

使用 栈结构 保存历史状态（棋盘、阴影、步数），最多 50 步。

每执行一次有效移动前调用 pushUndoState() 压入当前状态，撤销时弹出栈顶并恢复。

2.5 计时与最佳记录

使用 clock() 函数，单位秒。

普通模式：首次移动时 startTiming()，胜利时 stopTiming() 并比较 bestTimeMap。

盲棋模式：分别记录观察阶段用时 (observeStart 到按 P) 和复原阶段用时（recoverStart 到提交），胜利后更新三个映射表（观察最佳、复原最佳、总最佳）。

五. 测试

5.1 测试环境

· 操作系统：Windows 10/11

· 编译器：GCC 9.3.0 + EGE 图形库

· 显示器分辨率：1920×1080（窗口 900×850 正常显示）

5.2 功能测试用例

1. 启动菜单　运行程序　显示菜单，默认选中 4×4
2. 选择棋盘大小 按 3 当前选中变成 3×3
3. 选择难度　按 1 进入游戏，棋盘 3×3 无阴影

4. 移动数字　按 WASD　空白格相邻数字移动，步数+1

5. 胜利判定　排成 1 2 3 / 4 5 6 / 7 8 空格 显 示 “ 胜利! ”特效，计时停止

6. 重置 按 R 回到初始盘面，步数清零

7. 撤销 移动几步后按 U 回退一步，步数恢复

8. 洗牌 按 P 随机生成新局，步数归零

9. 阴影效果　难度 3，4×4 约 30% 格子紫底 NK

10. 空格+阴影阴影出现在当前空格位置 红 底 白 色 NKU

11. 盲棋模式 菜单按 0，观察阶段按 P 复原阶段仅显示紫底 NKU

12. 盲棋提交 复原完成后按 P 若正确则胜利并记录时间

13. 盲棋失败 复原错误提交 显示真实盘面，禁用操作

14. 提示（H）　3×3 难度 1 未胜利时按 H 从 空 白 格画出红色箭头指向建议移动方向

15. 返回菜单 游戏中按 Q　跳回主菜单

16. 最佳记录 完成一局后重新菜单 菜 单 右 下 显 示 对 应 难度最佳时间

5.3 边界测试

· 最大棋盘 8×8：窗口自动适应格子大小，移动流畅，

无卡顿。

· 难度 6（仅一个非阴影）：正确显示其余全为阴影格。

· 连续撤销超过 50 步：栈自动丢弃最早历史，保留最近 50 步。

· 快速多次按提示：BFS 超时后自动回落贪心，不会阻塞界面。

· 盲棋模式下按 Q 重置后重新游戏：计时和状态正确重置。

5.4 性能测试

· 提示 BFS 搜索时间控制在 2 秒内，对 4×4 棋盘平均 <0.5 秒。

· 图形刷新稳定在 20 FPS，无内存泄漏。

六. 收获

通过本次数字华容道小游戏的开发，我取得了以下进步：

1. 深入理解 C++ 与图形编程

熟练使用 EGE 库的双缓冲绘图（setactivepage / setvisualpage）、字体设置、键盘交互，掌握了图形界面程序的基本框架。

2. 算法实践

曼哈顿距离作为启发式评估函数。

BFS 在状态空间搜索中的应用，并结合时间约束进行优化。

随机洗牌算法保证可解性。

3. 数据结构运用

vector<vector<int>> 存储棋盘、阴影状态。

stack<UndoStep> 实现撤销栈。

map<int, double> 保存不同尺寸/难度的最佳记录。

unordered_set<string> 用于 BFS 去重。

4. 游戏机制设计

设计多级难度与阴影系统，增加游戏挑战性。

盲棋模式的设计与时间分段记录，提高了程序逻辑的复杂性。

提示系统兼顾速度与最优性。

5. 调试与测试能力

通过日志输出和断点调试解决了 BFS 超时、阴影显示错误、盲棋状态转换等 bug。

编写测试用例，覆盖主菜单、游戏、盲棋、边界情况，保证程序鲁棒性。

6. 用户体验优化

添加烟花胜利特效、最佳记录展示、实时计时步数显示。

键盘响应及时，操作反馈清晰。

菜单中提供完整游戏规则说明。

总之，本项目是一次完整的 C++ 游戏开发实践，融合了算法、数据结构、图形界面和交互设计，极大提升了我的工程能力和解决问题的能力。